

포스팟 근무 시 담당한 프로젝트

PORTFOLIO

2023



사진과 영상, 그리고 코딩을
좋아하는 개발자
서희정입니다.

개인정보

생년월일	1994. 05. 20
전화번호	010-9937-4928
이메일	heejung0520@naver.com
학교/전공	장안대학교/멀티미디어콘텐츠과

주요 경력사항

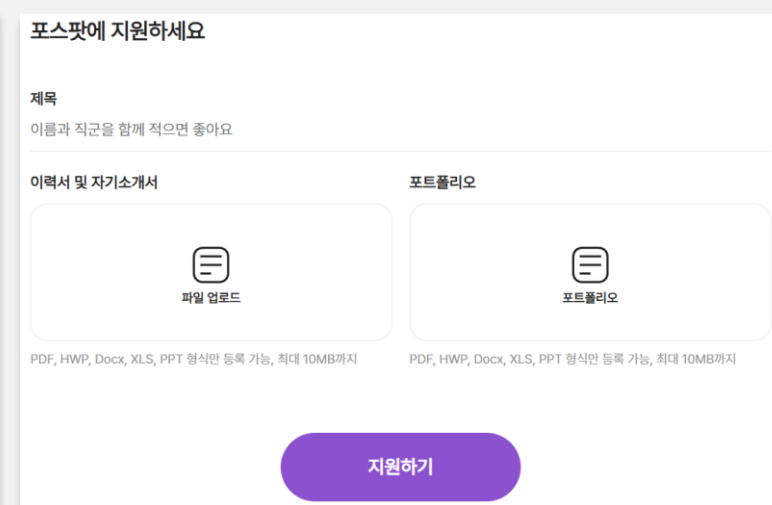
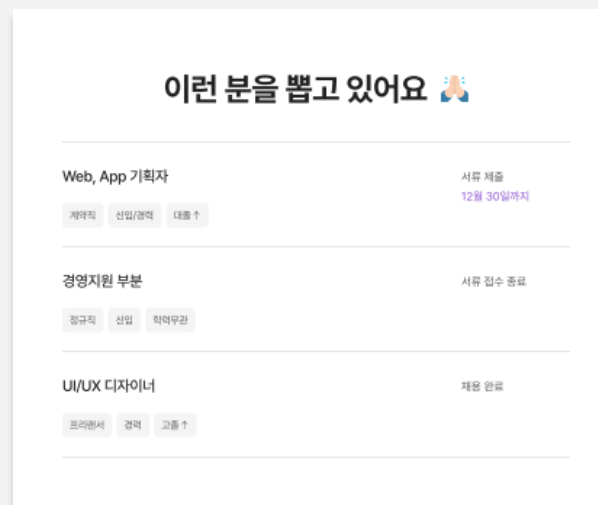
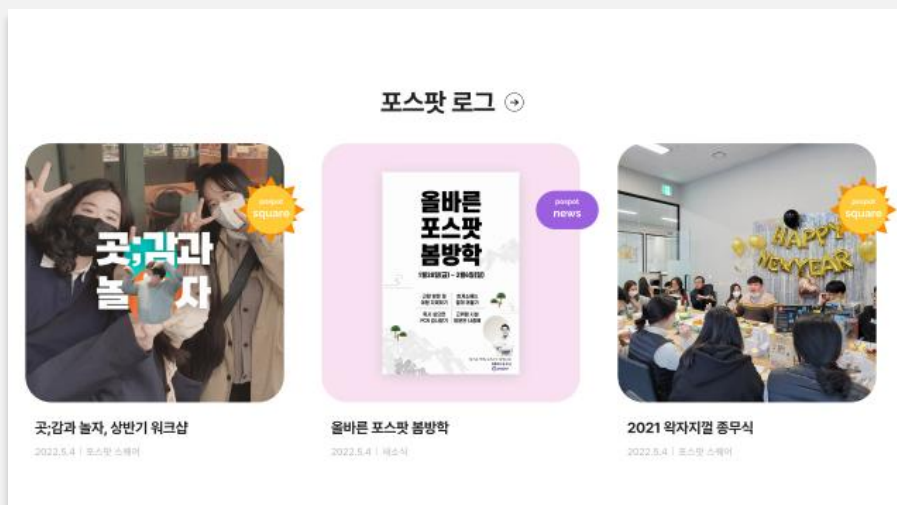
2014 - 2017	장안대학교 멀티미디어콘텐츠과 졸업
2017 - 2019	두리웹 / 마케팅부서 근무
2019 - 2020	NHN / 클라우드 지원팀 근무
2020 - 2020	클래스101 / 101Production 근무
2021 - 2023	포스팟 / 개발부서 근무
2023 - 2023	플리토, 태성콘텍 프리랜서 근무

Programing skill

Spring Boot	
Node.js	
HTML/CSS	
MariaDB	
Oracle	
MongoDB	

자격사항

2014	Office Excel® 2010 Office PowerPoint® 2010
2015	GTQ 일러스트 1급
2016	리눅스 마스터 2급
2019	자동차 운전면허 2종 ACA Photoshop cs6 ACA Premiere cs6
2021	COS PRO 2급 (Java)
2022	SQL 개발자 (SQLD)



활동 내용 DB 스키마 설계 (포스팟 로그, 채용 공고)
Admin 페이지 생성 (core ui)
API 작업 및 데이터 연동

기술 스택 React, nodeJS, mysql

내용

- React의 coreUI 템플릿을 이용하여 Admin 페이지를 생성하여 관리자가 효율적으로 웹사이트를 관리하고, 사용자들의 요구에 신속하게 대응할 수 있도록 지원


```
// 미들웨어 추가
app.use(cors({
  origin: true,
  credentials: true
})))
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
```

```
// 기본 설정으로 여러개의 파일 업로드를 처리
exports.fileUpload = multer({
  storage: multer.diskStorage({
    destination(req, file, cb) {
      cb(null, "./recruitResume/"); // 파일의 저장 위치 설정
    },
    filename: (req, file, cb) => {
      cb(null, file.originalname); // 파일이 저장될 때 이름 설정
    }
  }),
  fileFilter: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    if(ext === '.pdf' || ext === '.hwp' || ext === '.docx' || ext === '.xls'
      || ext === '.PDF' || ext === '.HWP' || ext === '.DOCX' || ext === '.XLS') {
      req.fileValidationError = null;
      cb(null, true);
    } else {
      req.fileValidationError = "파일 형식이 올바르지 않습니다.";
      cb(null, false);
    }
  },
  limits: { fileSize: 10 * 1024 * 1024 }
});
```

```
// 입사지원 파일 첨부 메일 전송 (npm install nodemailer)
exports.resumeFileSendMail = async (req, res) => {
  const folderPath = "./recruitResume/";
  // 첨부파일 리스트
  let matchedFileList = [req.files.resumeFile[0].originalname];
  // 경원지원 같은 직군일 경우 포트폴리오가 없을 수 있기 때문에 추가 22.05.12
  if(typeof req.files.portfoliFile !== "undefined" ) {
    matchedFileList.push(req.files.portfoliFile[0].originalname)
  }
  let recruitTitle = req.body.recruitTitle;
  if(typeof recruitTitle === "undefined" ) {
    recruitTitle = '포스팟'
  }

  const emailSubject = `[입사지원] ${req.body.mailTitle}`;
  const emailHtml = `
<div style="width: 500px; border-top: 4px solid #8B51CF; margin: 30px; padding: 30px 0 0 0;">
  <h1 style="margin: 0; padding: 0 5px; font-size: 28px; font-weight: 400;">
    <span style="color: #8B51CF;">${recruitTitle}</span> 입사지원 </h1>
  <p style="line-height: 26px; margin-top: 50px; padding: 0 5px;">
    안녕하세요.<br />
    채용공고 <b style="color: #8B51CF;">${recruitTitle}</b> 입사지원 메일입니다.<br /><br />
    <b style="color: #8B51CF;">[제목] </b> ${req.body.mailTitle}<br />
    <b style="color: #8B51CF;">[이력서 및 자기소개서] </b> ${matchedFileList[0]}<br />
    <b style="color: #8B51CF;">[포트폴리오] </b> ${typeof req.files.portfoliFile !== "undefined" ?
    감사합니다.<br />
  </p>
  <div style="border-top: 1px solid #000; padding: 10px 5px;">
    <a style="font-size: 13px; color: #8B51CF; text-align: center;" href="${req.body.pageURL}">
  </div>
</div>
`
```

백엔드

- cors 모듈을 사용해 다른 도메인인 Front와 Back의 리소스 요청 허용
- 서버에 파일 및 이미지를 업로드 할 수 있게 multer 모듈을 사용
- NodeMailer 모듈을 활용하여 입사지원서를 회사 홈페이지에서 담당자 메일로 바로 보낼 수 있게 구현

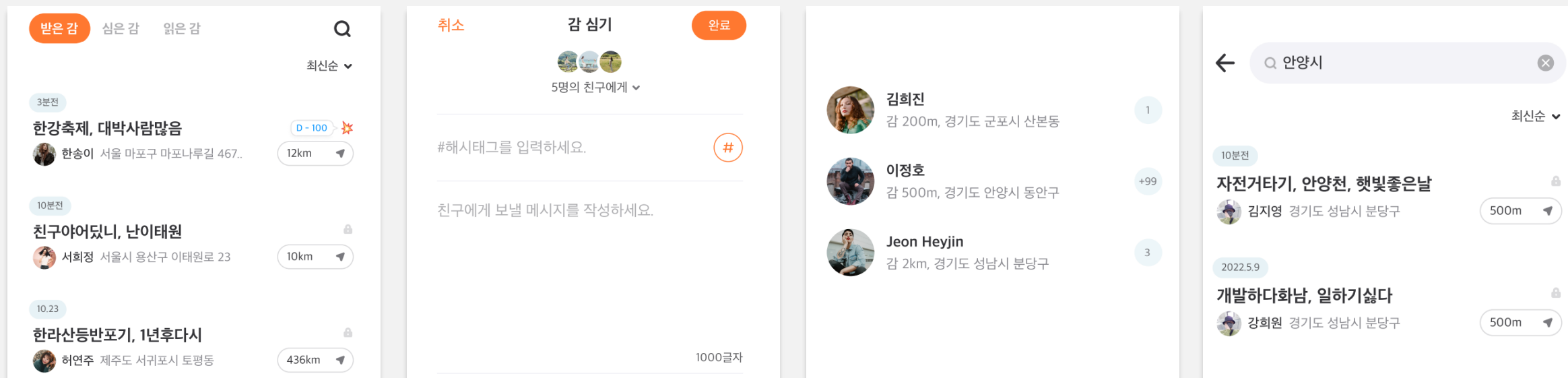
```
// 토큰 체크
useEffect(() => {
  axios({
    method: 'get',
    url: process.env.REACT_APP_API+'api/loginprocess',
    withCredentials: true,
    headers: {
      // localStorage에 있는 토큰을 헤더에 넣어서 api로 전송
      Authorization: localStorage.getItem('jwtoken')
    }
  }).then(res => {
    if (res.data.code === 502) {
      // 토큰 인증 실패
      alert(res.data.msg);
      return window.location.replace("/login");
    } else {
      setRecruit((prevState) => {
        return { ...prevState, deadline: submitDate }
      });
      // 토큰이 유효하다면 화면 보이게 설정
      setShow(true);
    }
  }).catch(error => {
    console.log(error);
  });
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);
```

```
// 채용공고 데이터 API로 전송
const onSubmit =(e)=> {
  console.log(recruit);

  if (window.confirm("저장하시겠습니까?") === true) {
    axios({
      method: 'POST',
      url: process.env.REACT_APP_API+'api/addRecruit',
      withCredentials: true,
      data: recruit
    })
    .then(res => {
      alert(res.data.msg);
      return window.location.replace("/recruit");
    })
    .catch(error => {
      console.log(error);
      return false;
    });
  } else {
    return false;
  }
}
```

프론트

- 관리자만 접근을 허용하기 위해 로그인 시 토큰을 제공하여 프론트에서 localStorage에 저장된 토큰 인증
- 저장할 데이터를 서버로 보내기 위해 axios 모듈 사용

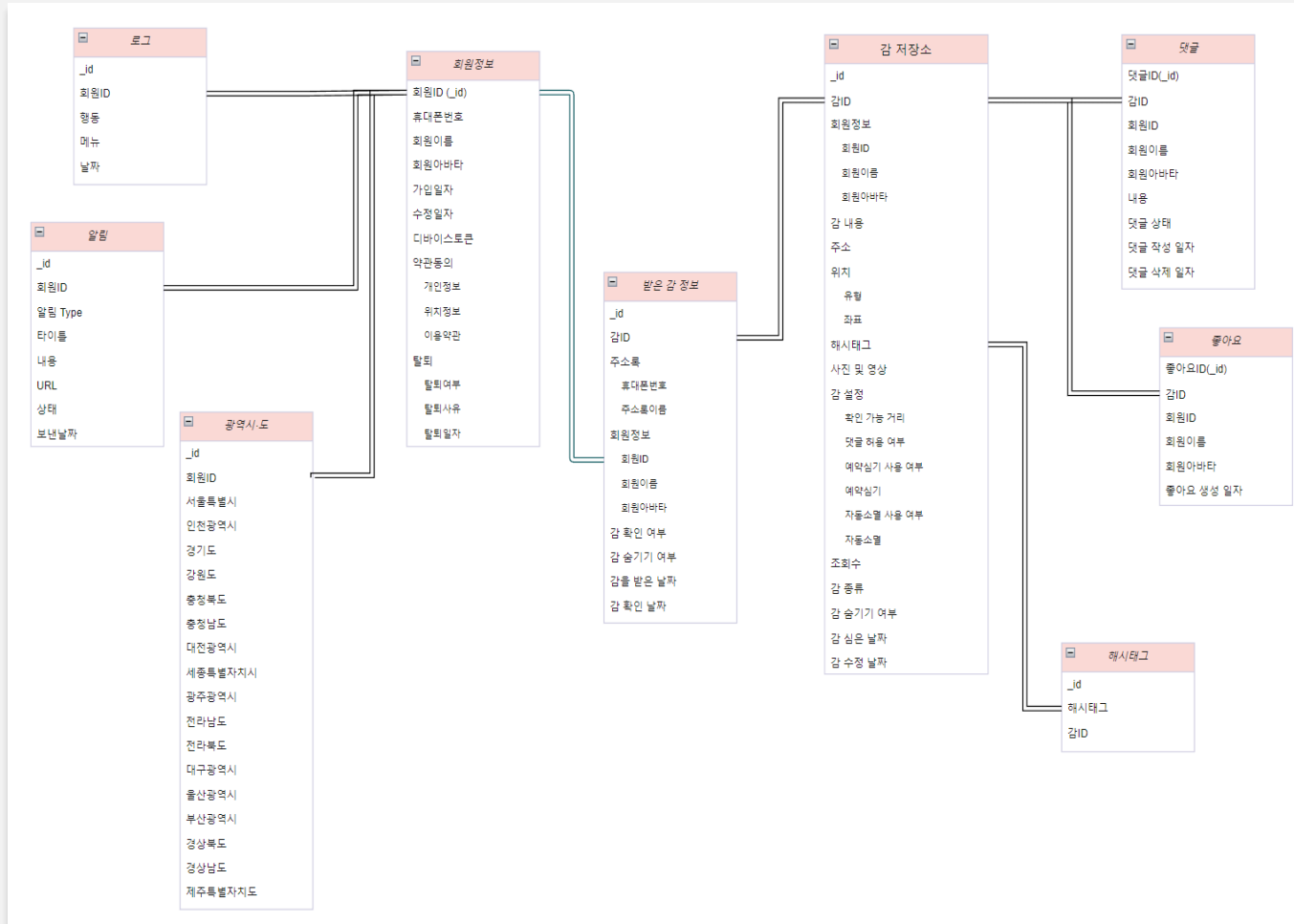


업무내용 데이터 분석 및 기능 분석
DB 스키마 설계
GraphQL API 및 함수 개발

기술스택 mongoDB, GraphQL

내용

- geoNear 파이프라인으로 사용자의 위치에 가장 가까운 게시물을 효율적으로 검색하고 제공
- GraphQL을 활용하여 클라이언트가 필요한 데이터만을 제공하여 클라이언트가 필요 이상의 데이터를 받아들이는 오버페칭 문제와 필요한 데이터를 한 번에 받지 못하는 언더페칭 문제를 해결
- mongodb atlas로 서버리스 구축
- 프론트엔드를 위해 총 42개의 graphql 쿼리 생성



DB스키마설계

- 유연한 데이터 관리를 위해 nosql 사용
- 총 9개의 스키마 생성
- 각각의 기능에서 데이터 update가 많기 때문에 스키마를 관계형 데이터베이스처럼 설계

메뉴	감저장소	회원ID	감저장소 개수	gam_received	query { gamReceived_notConfirm (input: {loginID:"62b9388cb548442ca6a7902a"}) { notConfirmCnt } }	O	\$count	확인 안한 감 개수 반환
----	------	------	---------	--------------	--	---	---------	---------------

GraphQL API 및 함수 개발

- 하나의 엔드포인트를 가지는 graphql API 구축
- mongoDB atlas에서 제공하는 기본 Resolvers 외 필요한 함수 추가
- 클라이언트와 서버간의 데이터가 정확해야하기 때문에
명확하게 정의된 스키마 선언

```
// 감저장소 조회 조건
let storage_aggregation = [
  {
    "$geoNear": {
      "query": geoNear_query,
      "near": {
        "type": "Point",
        "coordinates": [arg.lng, arg.lat]
      },
      "distanceField": "distance",
      "key": "location.coordinates",
    }
  },
  {
    "$sort": { "createdAt": -1 }
  },
  {
    "$lookup": {
      "from": 'gam_received',
      "localField": 'gamID',
      "foreignField": 'gamID',
      "as": 'receivedGam'
    }
  },
  {
    "$addFields": {
      // 열린감
      "gamOpen": {
        "$cond": {
          if: {
            "$gt": [{"toInt": "$gamSetting.setConfirmableDistance"}, "$distance"]
          },
          then: true,
          else: false
        }
      }
    }
  },
]
```

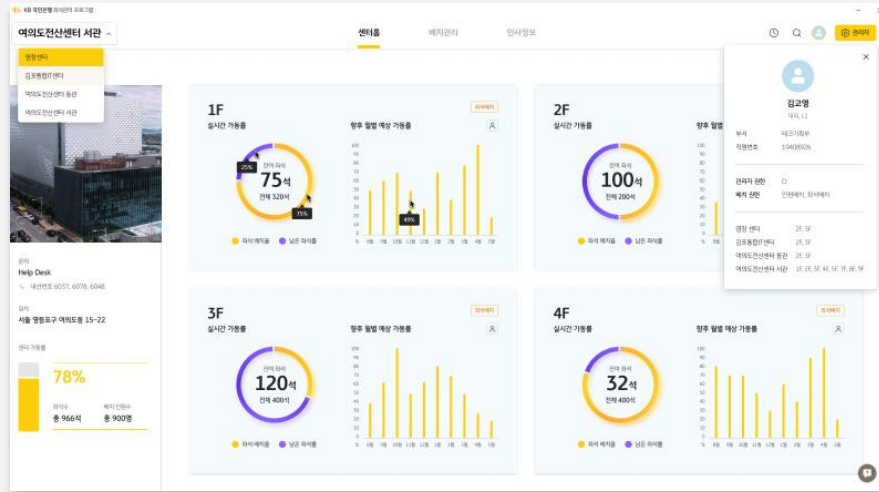
```
{
  "field_name": "gam_create",
  "function_name": "gamCreate",
  "input_type": {
    "properties": {
      "contacts": {
        "bsonType": "array",
        "items": {
          "bsonType": "object",
          "properties": {
            "contactName": {
              "bsonType": "string"
            },
            "userHP": {
              "bsonType": "string"
            }
          }
        }
      },
      "title": "contacts"
    },
    "gamInfo": {
      "properties": {
        "_id": {
          "bsonType": "objectId"
        },
        "address": {
          "bsonType": "string"
        },
        "createDate": {
          "bsonType": "date"
        },
        "gamContent": {
          "bsonType": "string"
        }
      }
    }
  }
}
```

국민은행 좌석배치 프로젝트

2022.08 - 2023.02

07

상주 인력의 좌석 이동 및 업무 공간 관리 프로그램



구분	좌석번호	좌석명	좌석상태	좌석위치	좌석면적	좌석면적비율	좌석면적비율비율	좌석면적비율비율비율	좌석면적비율비율비율비율	좌석면적비율비율비율비율비율	좌석면적비율비율비율비율비율비율
1F	1000	사무실	사용	1F-1000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
2F	2000	사무실	사용	2F-2000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
3F	3000	사무실	사용	3F-3000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
4F	4000	사무실	사용	4F-4000	1.0	1.0	1.0	1.0	1.0	1.0	1.0

구분	좌석번호	좌석명	좌석상태	좌석위치	좌석면적	좌석면적비율	좌석면적비율비율	좌석면적비율비율비율	좌석면적비율비율비율비율	좌석면적비율비율비율비율비율	좌석면적비율비율비율비율비율비율
1F	1000	사무실	사용	1F-1000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
2F	2000	사무실	사용	2F-2000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
3F	3000	사무실	사용	3F-3000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
4F	4000	사무실	사용	4F-4000	1.0	1.0	1.0	1.0	1.0	1.0	1.0

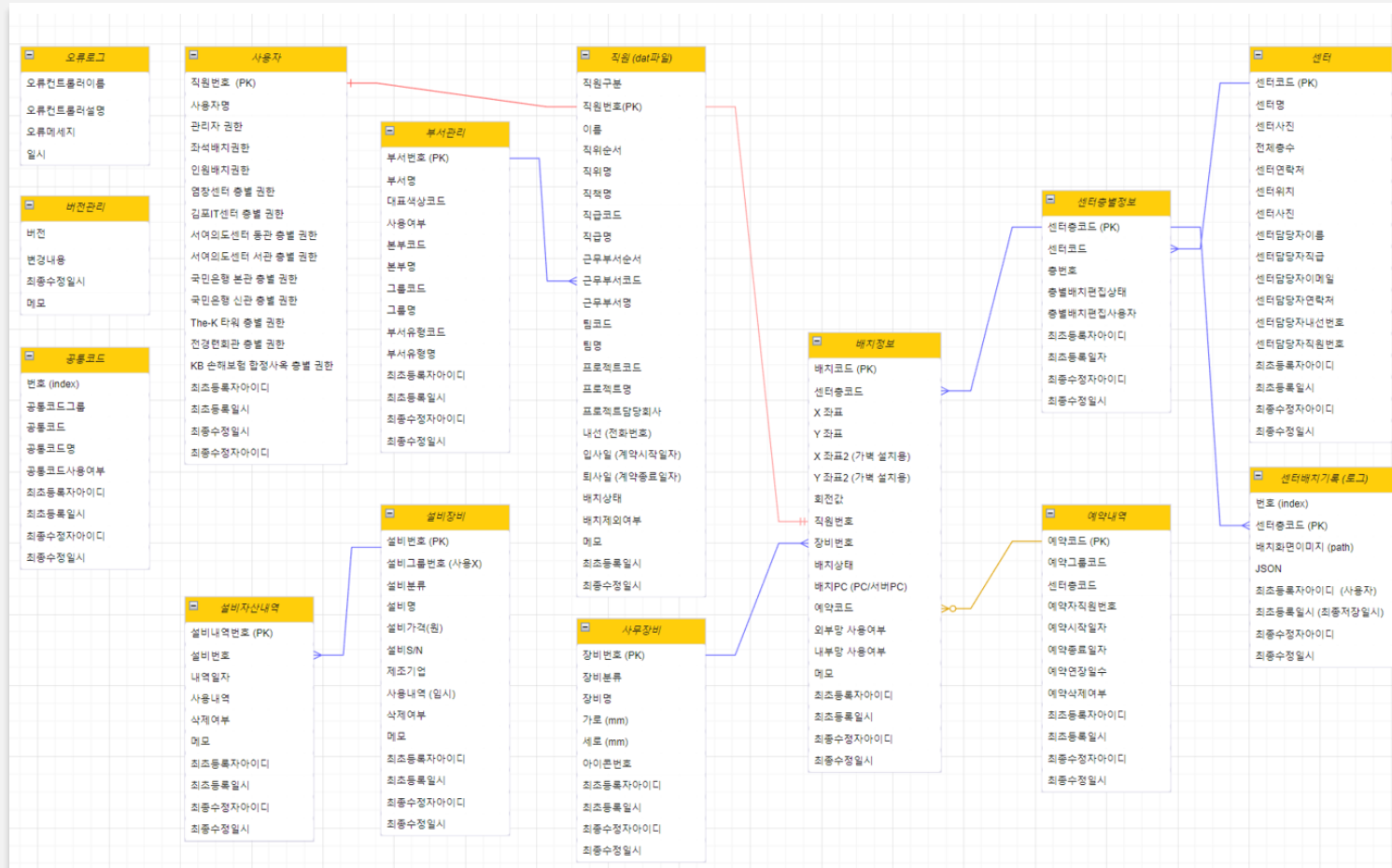
구분	좌석번호	좌석명	좌석상태	좌석위치	좌석면적	좌석면적비율	좌석면적비율비율	좌석면적비율비율비율	좌석면적비율비율비율비율	좌석면적비율비율비율비율비율	좌석면적비율비율비율비율비율비율
1F	1000	사무실	사용	1F-1000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
2F	2000	사무실	사용	2F-2000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
3F	3000	사무실	사용	3F-3000	1.0	1.0	1.0	1.0	1.0	1.0	1.0
4F	4000	사무실	사용	4F-4000	1.0	1.0	1.0	1.0	1.0	1.0	1.0

업무내용 DB 스키마 설계
프로젝트 전체 API 개발
Docker 서버 배포

기술스택 Spring Boot, Mybits, NodeJS,
Oracle, Docker

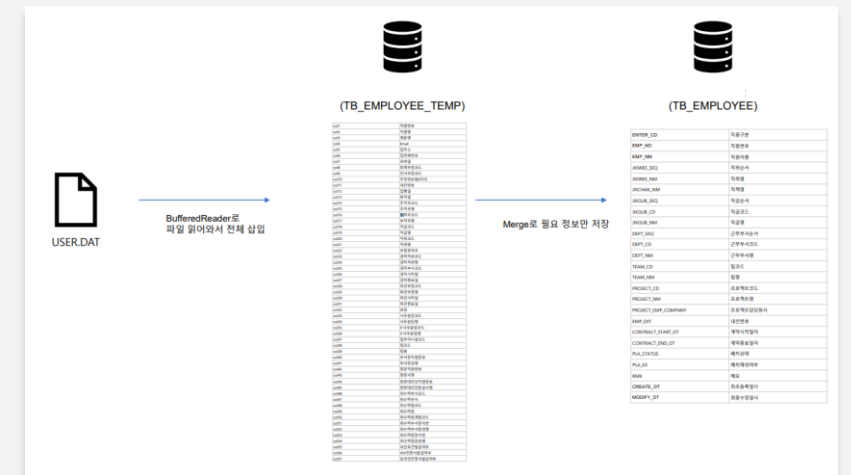
내용

- Spring Boot와 MyBatis를 활용하여 프로젝트 전체 API 개발 및 DB 스키마 설계
- KB 내부 직원 테이블과 새로 설계한 좌석 테이블을 Join한 좌석 배치 비율 쿼리를 생성하여 실시간으로 좌석 배치 상황을 파악 가능
- Node.js를 활용하여 하루에 한 번씩 dat파일을 DB에 적재하는 스케줄러를 생성하여 최신 정보에 기반한 서비스 제공 가능
- Docker를 활용하여 서버를 배포하여 개발 환경과 운영 환경의 일관성 유지



DB 스키마 설계

- 총 14개의 스키마 생성
- 직원 테이블은 KB 내부 직원 테이블에서 필요한 데이터만 적재 (1차)
- 직원 정보를 dat파일에서 읽어와서 임시 테이블에 전체 삽입 후 merge로 필요한 데이터만 저장 (2차)
- 데이터 생성, 수정, 삭제가 많기 때문에 외래키를 사용하지 않고 Join을 사용하여 테이블을 연결



상주 인력의 좌석 이동 및 업무 공간 관리 프로그램

```
// 사용자 리스트
@GetMapping(value = "/admin/userList")
public List<userListVO> selectAdminUserList(@RequestParam(required = false, value = "page", defaultValue= "1") int page) {
    System.out.println("### selectAdminUserList ###");
    List<userListVO> adminUserList = new ArrayList<userListVO>();
    try {
        adminUserList.addAll(adminService.selectAdminUserList(page));
    } catch (Exception e) {
        commonService.insertErrLog("selectAdminUserList", "사용자 리스트", e.toString());
    } finally {
        return adminUserList;
    }
}
```

```
<mapper namespace="com.kb.seating.api.dao.adminDao">
<!-- 사용자 리스트 -->
<select id="selectAdminUserList" parameterType="int" resultType="userListVO">
WITH kbEMP AS (
SELECT
    tEmp.EMP_NM, -- 직원이름
    tEmp.DEPT_SEQ,
    tEmp.JIKWEE_SEQ,
    tEmp.JIKGUB_SEQ,
    NVL2(tEmp.JIKWEE_NM, CONCAT(tEmp.JIKWEE_SEQ, CONCAT('-', CONCAT(NVL(tEmp.JIKCHAK_NM, '-'), ''))),
    tEmp.JIKGUB_SEQ, -- 직급
    tEmp.DEPT_NM AS DEPT_NM, -- 근무부서
    tEmp.EMP_NO AS EMP_NO -- 직원번호
FROM C##KBPROJECT.TSHRMSM11 tEmp
WHERE tEmp.ENTER_CD = 'KB'
)
SELECT ROW_NUMBER() OVER(ORDER BY t.AUTH_ADMIN_SEQ DESC ,
    (CASE WHEN t.AUTH_PLA_SEAT = 'Y' THEN 1 ELSE 2 END) DESC,
    (CASE WHEN t.AUTH_PLA_EMP = 'Y' THEN 1 ELSE 2 END) DESC,
    CONVERT(t.DEPT_NM, 'ISO2022-KR') DESC, t.DEPT_SEQ DESC,
    t.JIKWEE_SEQ DESC, t.JIKGUB_SEQ DESC, CONVERT(t.EMP_NM, 'ISO2022-KR') DESC)
    rn, t.*
FROM
(
SELECT
    NVL2(tEmp.EMP_NM, tEmp.EMP_NM , tu.USER_NM) AS EMP_NM, -- 직원이름
    tEmp.DEPT_SEQ,
    tEmp.JIKWEE_SEQ,
```

```
// 사용자 리스트
@Override
public List<userListVO> selectAdminUserList(int page) {
    return adminDao.selectAdminUserList(page);
}
```

API

- API 생성 순서
 - 필요한 데이터를 담을 VO 생성
 - DB 쿼리를 실행할 Mapper 작성 (~DaoImpl.xml)
 - Mapper를 실행할 DAO 작성 (~Dao.java)
 - DAO를 호출할 Service 인터페이스 작성 (~Service.java)
 - Service implements 후 필요 내용 작성(~ServiceImpl.java)
 - Service를 호출할 Controller 작성 (~Controller.java)
 - Controller에 API 주소 작성
- API 실행 순서
 - API 호출 -> Controller에서 Parameter parsing
 - Controller : Service 호출
 - Service : 필요한 DAO 호출
 - DAO : 해당 mapper 호출하여 데이터 타입(ex. vo)에 맞춰 Service에 반환
 - Service : Controller로 보낼 데이터 정리 후 반환
 - Controller : 오류가 없는지 확인하고 반환

상주 인력의 좌석 이동 및 업무 공간 관리 프로그램

```
// 0 12 1-31 * *
// 스케줄러를 설정하고 바로 시작 (실행 주기, 콜백함수)
var dat = cron.schedule('40 14 * * *', function () {
  fetch("https://kbseatapi.anypot.co.kr/dat", requestOptions)
    .then(
      response => response.text()
    )
    .then(result => console.log('dat | ' + result + ' | ' + new Date()))
    .catch(error => console.log('error', error));
});

var merge = cron.schedule('50 14 * * *', function () {
  fetch("https://kbseatapi.anypot.co.kr/dataMerge", requestOptions)
    .then(
      response => response.text()
    )
    .then(result => console.log('merge | ' + result + ' | ' + new Date()))
    .catch(error => console.log('error', error));
});
```

```
<update id = "commonDataMergeDept">
MERGE

      INTO C##KBPROJECT.TSHRMSM07 dept

USING (
SELECT DISTINCT *
FROM (
      SELECT
        substr(col1,1,10) AS DEPT_NO,
        substr(col2,1,4) AS GL_CD,
        substr(col3,1,100) AS DEPT_NM,
        substr(col9,1,6) AS BONBU_CD,
        substr(col10,1,100) AS BONBU_NM,
```

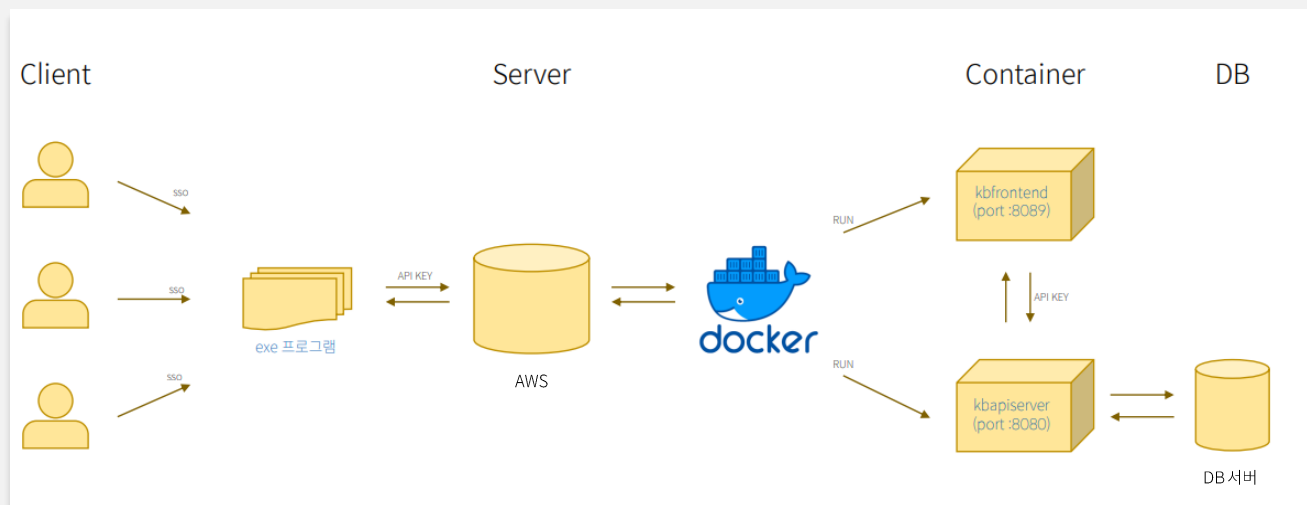
스케줄러

- 매일 업데이트 되는 직원 정보 dat파일을 읽어와서 해당 테이블에 넣는 작업을 하는 API를 매일 같은 시간에 호출하기 위해 nodeJS로 스케줄러 작업
- 무중단 서비스인 pm2로 nodeJS를 관리

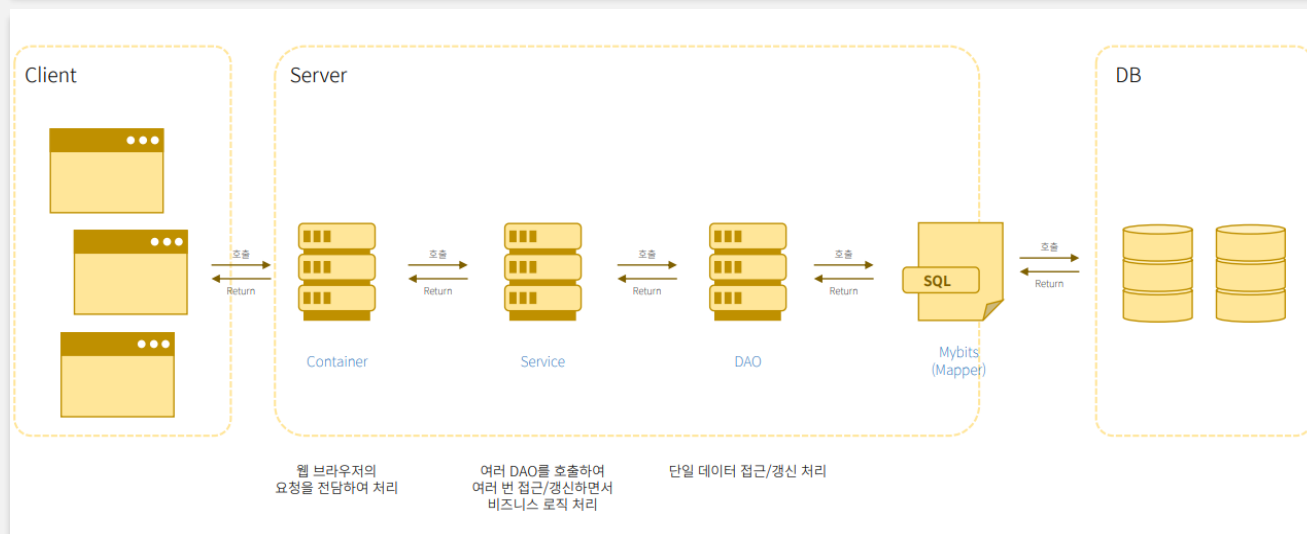
```
/* 임시테이블에서 실제 테이블로 merge*/
@GetMapping(value = "/dataMerge")
public String commonDataMerge() {
  LocalDate now = LocalDate.now(ZoneId.of("Asia/Seoul"));
  String result = "";
  try {
    System.out.println("### commonDataMerge ### ");
    result = commonService.commonDataMerge();
    cm.logger.info("[commonDataMerge] " + now + " 임시테이블에서");
  } catch (Exception e) {
    commonService.insertErrLog("commonDataMerge", "임시테이블에서");
  } finally {
    return result;
  }
}
```

상주 인력의 좌석 이동 및 업무 공간 관리 프로그램

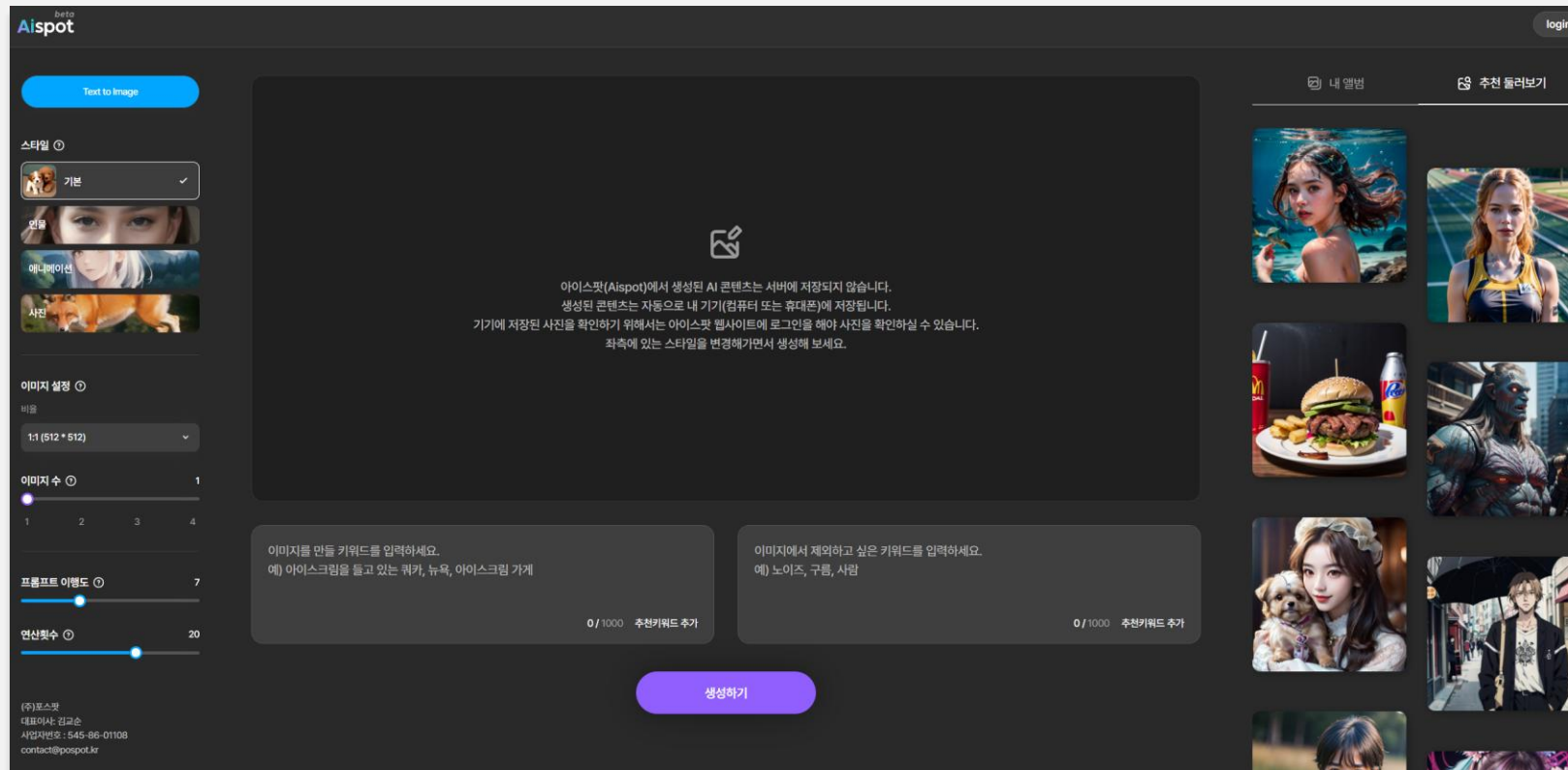
도커 시스템 구성도



Spring boot 서비스 구조



스테이블 디퓨전 기반 이미지 생성 홈페이지 <https://www.youtube.com/shorts/KEsFHToQQZM?feature=share>



업무내용 DB 스키마 설계
전체 API 개발
프롬프트 구성

기술스택 nodeJS, MariaDB, MongoDB, graphql

내용

- 데이터의 효율적인 관리를 위해 회원 정보는 MariaDB, 생성된 이미지 정보는 MongoDB에 저장하도록 구성
- 정확한 이미지 생성을 위해 미리 프롬프트를 구성하여 사용자의 편의성을 향상

스테이블 디퓨전 기반 이미지 생성 홈페이지

```

/* GraphQL 명세에서 사용될 데이터 */
const typeDefs = [
  enums.typeDefs,
  queries,
  mutations,
  scalarType.typeDefs,
  logs.typeDefs,
  txt2img.typeDefs,
  img2img.typeDefs,
]

/* 서비스의 액션들을 함수로 지정, 요청에 따라 데이터를 반환, 입력, 수정 삭제 실행 */
const resolvers = [
  scalarType.resolvers,
  logs.resolvers,
  txt2img.resolvers,
  img2img.resolvers,
]

```

```

const resolvers = {
  Query: {
    I2Is: async (parent, args) => await I2IWorks.getI2IList(args),
    I2ISearch: async (parent, args) => {
      const I2ISearch = await I2IWorks.getI2IListEmail(args);
      return I2ISearch;
    },
    I2IDetail: async (parent, args) => {
      const I2IDetail = await I2IWorks.getI2IDetail(args);
      return I2IDetail;
    }
  },
  Mutation: {
    createI2I: async (parent, args) => {
      const insertedT2I = await I2IWorks.createI2I(args.I2IInput);
      return insertedT2I;
    },
  },
}

```

```

/* T2I 이미지 생성 후 데이터 저장 */
createT2I: async (T2IInput) => {
  try {
    const { T2ICollection, logsCollection, promptCollection, negative_promptCollection } = await connectMongoDB();

    // 문자열 문자 배열 (' ', ' ' or ' ', ' ')
    T2IInput = replacePromptData(T2IInput);

    // 클럭어 작업
    T2IInput = promptWordChk(T2IInput);

    // 구글번역 API 호출
    T2IInput = await promptTranslateText(T2IInput);

    // stable diffusion API 호출
    const response = await stableDiffusion(T2IInput);
    console.log('### stableDiffusiontxt2img complete ###');
    const info = insertInfoData(response, T2IInput);
    const T2I = await T2ICollection.insertOne(info);

    // prompt 단어 로그 저장
    const new_prompts = insertPromptData(info.all_prompts);
    for (let prompt of new_prompts) {
      const existingPrompt = await promptCollection.findOne({ prompt });
      if (existingPrompt) {
        await promptCollection.updateOne({ prompt }, { $inc: { count: 1 } });
      } else {
        await promptCollection.insertOne({ prompt, count: 1 });
      }
    }

    // negative_prompt 단어 로그 저장
    if (info.all_negative_prompts) {
      const new_negative_prompts = insertNegativePromptData(info.all_negative_prompts);
      for (let prompt of new_negative_prompts) {
        const existingPrompt = await negative_promptCollection.findOne({ prompt });
        if (existingPrompt) {
          await negative_promptCollection.updateOne({ prompt }, { $inc: { count: 1 } });
        } else {
          await negative_promptCollection.insertOne({ prompt, count: 1 });
        }
      }
    }

    // log insert
    T2IInput.log.logDate = new Date();
    const log = await logsCollection.insertOne(T2IInput.log);
    delete info.images;
    return { images: response.data.images, info: info };
  } catch (error) {
    logger.log('error', 'createT2I failed to execute query: ${error}');
  }
}

```

graphql

- Node.js에서 사용하는 Express와 역할이 비슷한 Apollo 서버를 활용해 GraphQL API를 제공하는 서버를 개발
- 서버에 어떻게 데이터를 요청할지 정의한 스키마와 사용자가 쿼리를 요청했을 때 서버가 어떻게 처리할지 정의한 리졸버 선언
- 사용자가 입력한 텍스트를 기반으로 생성된 이미지를 저장

```
/* Session 모델 생성 */
const Session = sequelize.define('sessions', {
  sid: {
    type: Sequelize.STRING,
    primaryKey: true
  },
  expires: Sequelize.DATE,
  data: Sequelize.TEXT,
  createdAt: Sequelize.DATE,
  updatedAt: Sequelize.DATE,
  originalMaxAge: Sequelize.INTEGER
}, {
  tableName: 'sessions',
  underscored: true,
  timestamps: false
});
```

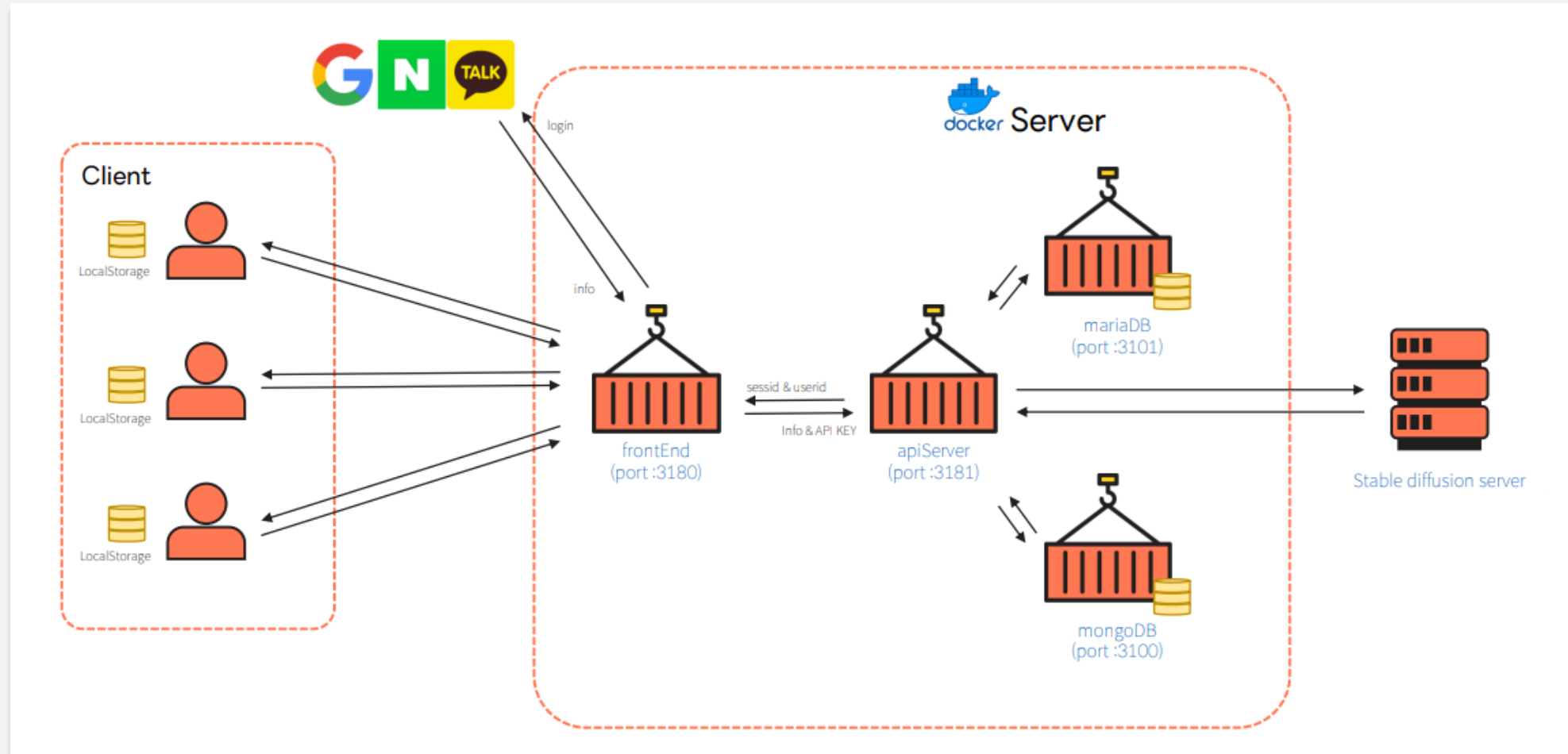
```
// DB에서 email을 이용해 유저 정보 조회 및 로그인 로직 추가
const exUser = await sequelize.query(
  `SELECT userID FROM user WHERE userEmail = ? AND password = ?`,
  {
    replacements: [userEmail, password],
    type: QueryTypes.SELECT,
  }
);
```

시퀀라이즈

- nodeJS에서 사용하는 ORM인 Sequelize로 MySQL을 연결하여 테이블 자동 생성
- 이후 쿼리에 서브쿼리를 사용할 가능성이 있기 때문에 Sequelize에서 raw 쿼리 사용

스테이블 디퓨전 기반 이미지 생성 홈페이지

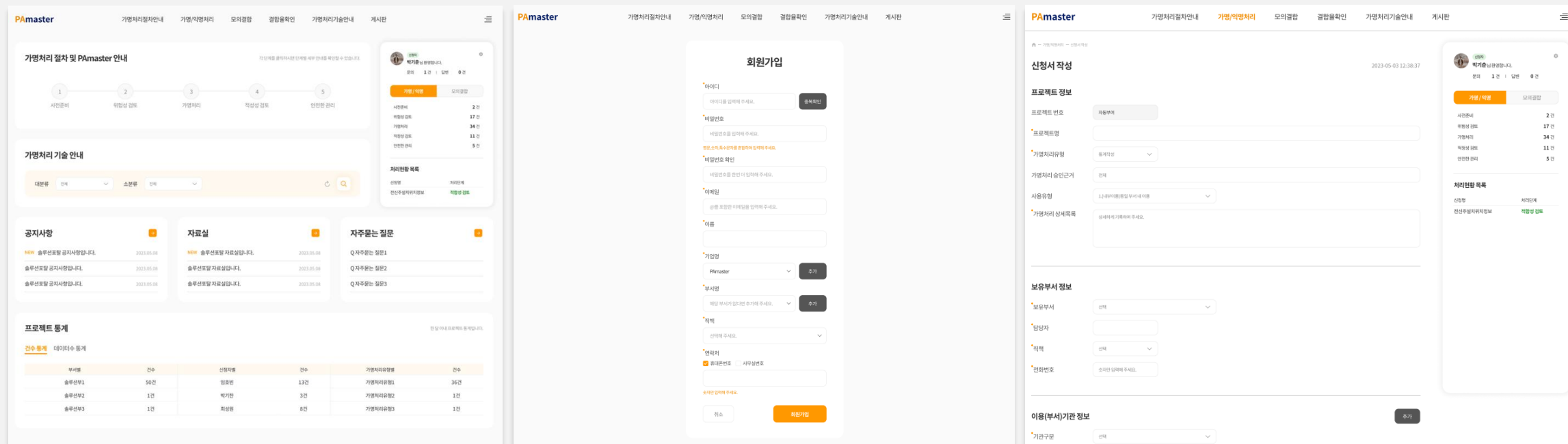
도커 시스템 구성도



솔루션포탈 2023.05

위치기반 SNS 플랫폼 프로젝트

16



업무내용 퍼블리싱

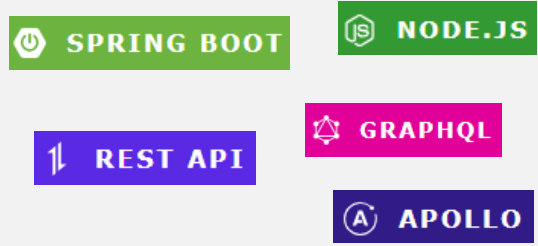
기술스택 HTML, CSS, JS

내용

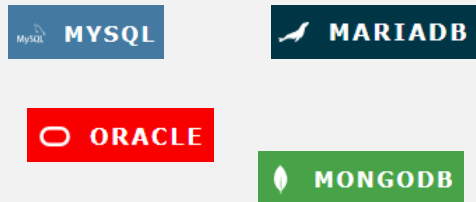
- 총 9페이지 퍼블리싱
- 라디오 버튼에 커스텀 스타일을 적용
- 메인페이지를 카드 형식으로 구성

Tech Stack

백엔드



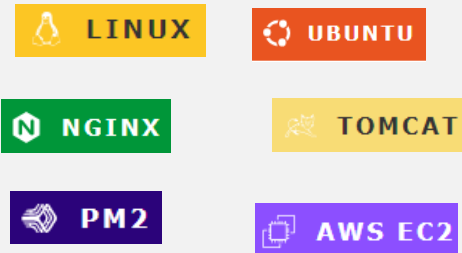
데이터베이스



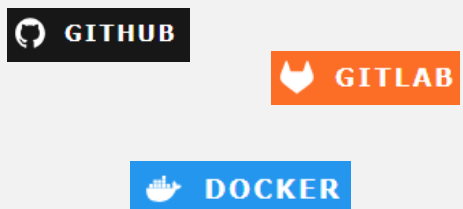
프론트엔드



인프라



데브옵스

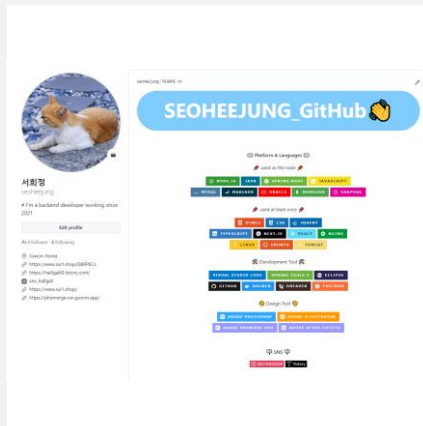


테스팅툴



저를 더 알고 싶으시다면.

GitHub



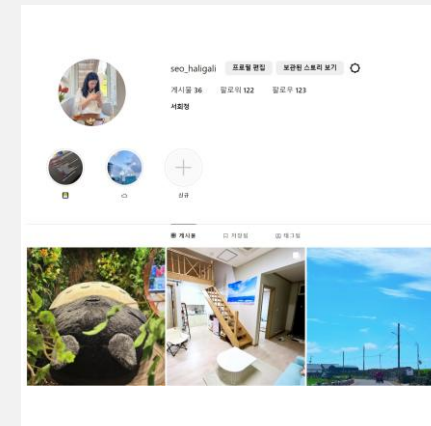
<https://github.com/seoheejung>

Notion



<https://galijung.notion.site/heejung-book-585550ea881b4f2eb8f110b0b27af2be>

Instagram



https://www.instagram.com/seo_haligali/

Other

| 감사합니다.